

Multivariate Telemetry Anomaly Detection & Monitoring System

Contents

Multivariate Telemetry Anomaly Detection & Monitoring System	4
Table of Contents	4
1. Problem Statement	4
Business / Operational Question	5
Intended Users	5
Potential Operational Actions	5
1.1 Why Anomaly Detection?	5
1.2 Project Objective	6
2. Project Motivation	6
2.1 The Monitoring Principle	6
2.2 Why Compare Multiple Detection Strategies?	7
2.3 Project Track	7
3. Architecture	8
3.1 Detection Pipeline	9
3.2 No Random Shuffle of the Time Series	9
4. Dataset	10
4.1 CATS — Controlled Anomalies Time Series	10
4.2 Dataset Characteristics	10
4.3 Variable Groups	10
4.4 Ground-Truth Labels	11
y	11
category	11
4.5 Anomaly Metadata	11
4.6 Normal Training Period	12
4.7 Missing Data	12
4.8 Dataset Integrity Rules	13
5. Project Structure	13
6. Setup & Installation	15
6.1 Prerequisites	15
6.2 Clone the Repository	15
6.3 Create a Virtual Environment	16
Windows	16
Linux / macOS	16
6.4 Install Dependencies	16
6.5 Configure Environment Variables	16
6.6 Dataset Acquisition	16
6.7 Start Jupyter	17
6.8 Run the API Locally	17

6.9 Run Tests	17
6.10 Run with Docker	17
7. Methodology	17
7.1 Step 1 — Problem Definition	18
7.2 Step 2 — Data Audit	19
7.3 Step 3 — Temporal Integrity Audit	19
7.4 Step 4 — Normal Behaviour Analysis	19
7.5 Step 5 — Leakage Audit	20
7.6 Step 6 — Rule-Based Baseline	20
7.7 Step 7 — Rolling Statistical Detector	20
7.8 Step 8 — Isolation Forest	21
7.9 Step 9 — Threshold Analysis	21
7.10 Step 10 — Point-Level Evaluation	22
7.11 Step 11 — Event-Level Evaluation	22
7.12 Step 12 — Detection Delay	22
7.13 Step 13 — False-Alarm Analysis	23
7.14 Step 14 — Missing-Data Robustness	23
7.15 Step 15 — Drift Analysis	24
7.16 Step 16 — Error Analysis	24
7.17 Step 17 — Final Detector Selection	24
8. Baselines	25
8.1 Baseline 1 — Rule-Based Detector	25
8.2 Baseline 2 — Rolling Statistical Detector	25
8.3 Candidate 3 — Isolation Forest	25
8.4 Baseline Recording Card	26
9. Experiments	26
9.1 Experiment Matrix	26
9.2 One-Variable-at-a-Time Rule	27
9.3 Experiment Tracking	27
9.4 Optimization Decision Matrix	27
10. Results	28
10.1 Overall Detector Comparison	28
10.2 Event-Level Results	28
10.3 Anomaly Category Results	28
10.4 Missing-Data Robustness	29
10.5 Drift Results	29
11. Error Analysis	29
11.1 False Positives	29
11.2 False Negatives	29
11.3 Error Slicing	30
11.4 Detection Delay Analysis	30
11.5 Root-Cause and Affected-Variable Analysis	30
12. Deployment	31
12.1 Backend — FastAPI	31
12.2 Example <code>/predict</code> Response	31
12.3 Input Validation	31
12.4 Monitoring Logs	31
12.5 Event Aggregation	32
12.6 Containerisation	32
12.7 CI/CD — GitHub Actions	32
12.8 Deployment Target	32

13. Evaluation Contract	33
13.1 Acceptance Principle	33
13.2 Final Selection Rule	33
13.3 Failure Criteria	34
14. Limitations	34
14.1 Simulated Benchmark	34
14.2 Dataset-Specific Anomalies	34
14.3 Missing Data	34
14.4 Drift	35
14.5 Ground Truth	35
14.6 Anomaly Detection Is Not Failure Prediction	35
14.7 Root Cause Is Not Automatically Identified	35
15. Security Considerations	36
15.1 API Input Validation	36
15.2 Environment Variables	36
15.3 Logging	36
15.4 Dependency Security	36
15.5 Model Integrity	36
16. Future Work	37
Appendix A — Anomaly Detection Workflow	37
Appendix B — Final Project Scorecard	39
Problem Definition	39
Dataset	39
Leakage	40
Baselines	40
Machine Learning	40
Evaluation	40
Robustness	41
Reproducibility	41
Deployment	41
Monitoring	41
Documentation	41
Appendix C — Mathematics Reference	42
C.1 Precision	42
C.2 Recall	42
C.3 F1 Score	42
C.4 False-Positive Rate	42
C.5 False-Negative Rate	43
C.6 Rolling Mean	43
C.7 Rolling Standard Deviation	43
C.8 Standardised Deviation	43
C.9 Detection Delay	43
C.10 Mean Detection Delay	44
C.11 Event Detection Rate	44
C.12 False-Alarm Rate	44
C.13 Isolation Forest Concept	44
C.14 Drift	44
C.15 Core Monitoring Principle	45

Multivariate Telemetry Anomaly Detection & Monitoring System

Detect abnormal behaviour. Measure detection quality. Quantify false alarms and detection delay. Prove robustness. Deploy and monitor.

A production-oriented machine learning project for detecting abnormal behaviour in multivariate system telemetry. The project compares a **rule-based baseline**, a **rolling statistical detector**, and an **Isolation Forest** approach, then evaluates them using operationally meaningful measures such as **precision, recall, false-alarm rate, detection delay, event-level detection, robustness to missing data, and behaviour under drift**.

The system is evaluated on the publicly available **CATS — Controlled Anomalies Time Series** benchmark, a multivariate telemetry dataset representing a simulated complex dynamical system with deliberately injected and precisely annotated anomalous segments.

Table of Contents

1. [Problem Statement](#)
2. [Project Motivation](#)
3. [Architecture](#)
4. [Dataset](#)
5. [Project Structure](#)
6. [Setup & Installation](#)
7. [Methodology](#)
8. [Baselines](#)
9. [Experiments](#)
10. [Results](#)
11. [Error Analysis](#)
12. [Deployment](#)
13. [Evaluation Contract](#)
14. [Limitations](#)
15. [Security Considerations](#)
16. [Future Work](#)
17. [Appendix A — Anomaly Detection Workflow](#)
18. [Appendix B — Final Project Scorecard](#)
19. [Appendix C — Mathematics Reference](#)

1. Problem Statement

Industrial and infrastructure systems continuously generate telemetry from equipment, control systems, and environmental conditions.

Most observations represent normal operation. Actual abnormal events are comparatively rare, and their causes may not be known in advance.

This creates a monitoring problem:

Can we detect when a multivariate system begins behaving abnormally, identify the event early enough for investigation, and do so without generating an unacceptable number of false alarms?

The project treats anomaly detection as an **operational monitoring problem**, rather than simply a binary classification problem.

Business / Operational Question

Given the telemetry observed from a monitored system, is the system behaving consistently with its learned normal operating behaviour?

Intended Users

The primary users are:

- Operations teams
- Maintenance teams
- Reliability engineers
- Infrastructure monitoring teams
- System operators
- Data/ML engineers responsible for automated monitoring

Potential Operational Actions

A detected anomaly may trigger:

- Investigation by an operator
- Inspection of affected equipment or subsystem
- Additional telemetry collection
- Maintenance scheduling
- Escalation to a reliability team
- Temporary operating restrictions
- Incident creation
- Automated monitoring alerts

The project does **not** claim that an anomaly automatically means equipment failure.

An anomaly means:

Observed system behaviour has deviated sufficiently from the behaviour considered normal by the monitoring system.

The detected event therefore becomes a signal for investigation.

1.1 Why Anomaly Detection?

Traditional supervised classification requires representative labelled examples of each class.

In operational monitoring, this assumption is often unrealistic.

A system may have:

- millions of normal observations;
- relatively few abnormal events;
- new failure modes that were not present in historical training data;
- changing operating conditions;
- incomplete or noisy telemetry.

Anomaly detection is therefore useful because the monitoring system can primarily learn or characterize **normal behaviour** and identify observations that deviate from it.

1.2 Project Objective

The objective is to build and evaluate a monitoring pipeline that:

1. learns or characterizes normal system behaviour;
 2. detects anomalous observations;
 3. groups anomalous observations into operational events;
 4. measures how quickly anomalies are detected;
 5. quantifies false alarms;
 6. compares simple statistical monitoring against unsupervised ML;
 7. tests robustness to missing telemetry;
 8. investigates behaviour under distributional drift;
 9. exposes the detector through an API;
 10. records monitoring events for later analysis.
-

2. Project Motivation

This project follows a **measure-first engineering approach**:

Define the monitoring problem → establish a baseline → build the simplest valid detector → measure → compare → investigate failures → deploy → monitor.

The project deliberately avoids treating model accuracy as the sole definition of success.

A detector can have high recall and still be operationally useless if it generates excessive false alarms.

Similarly, a detector can have excellent precision but detect anomalies several minutes or hours after they begin.

The evaluation therefore considers both **statistical quality** and **operational usefulness**.

2.1 The Monitoring Principle

The central principle is:

A useful anomaly detector must identify meaningful deviations early while keeping false alarms manageable.

This produces several competing objectives:

Objective	Why it matters
Precision	Limits unnecessary investigations
Recall	Reduces missed anomalies
False-alarm rate	Measures monitoring burden
Detection delay	Measures how quickly the system responds
Event detection rate	Measures whether actual incidents are detected
Robustness to missing data	Measures reliability when telemetry is incomplete
Robustness to drift	Measures resilience to changing operating conditions

2.2 Why Compare Multiple Detection Strategies?

The project compares three approaches because increasingly complex ML models should not automatically replace simpler monitoring methods.

The comparison is:

```
Rule-based detector
      ↓
Simple operational baseline

Rolling statistical detector
      ↓
Adaptive statistical monitoring

Isolation Forest
      ↓
Unsupervised multivariate ML
```

The goal is not:

“Use ML because ML is better.”

The goal is:

Determine whether the additional complexity of an ML detector produces a meaningful improvement over simpler alternatives.

2.3 Project Track

Track B — Classical Machine Learning / Time-Series Anomaly Detection

The project does not require:

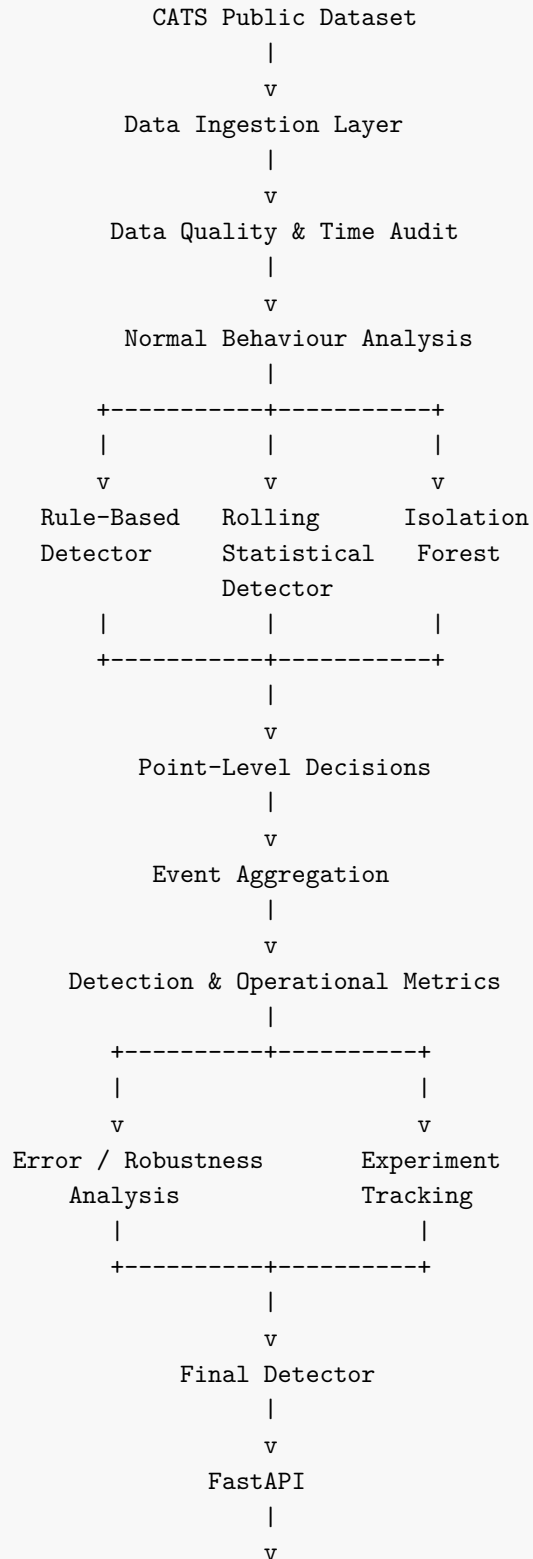
- LLMs
- RAG
- embeddings
- fine-tuning
- Transformers

The primary techniques are:

- Python
- pandas / NumPy
- statistical monitoring
- scikit-learn
- Isolation Forest
- time-series analysis
- FastAPI
- Docker
- pytest
- experiment tracking

- monitoring logs

3. Architecture



```
Monitoring Logs
|
v
Operational Review
```

3.1 Detection Pipeline

```
Raw telemetry
|
v
Timestamp validation
|
v
Data-quality checks
|
v
Feature preparation
|
v
Detector
|
v
Anomaly score / decision
|
v
Temporal event grouping
|
v
Alert
|
v
Log event
|
v
Evaluate detection performance
```

3.2 No Random Shuffle of the Time Series

The project preserves temporal ordering.

Randomly shuffling observations before evaluating a monitoring system can produce an unrealistic evaluation because future observations may influence the training or threshold-selection process.

The project therefore uses time-aware procedures for:

- normal-behaviour learning;
- validation;
- threshold selection;

- final evaluation;
- robustness experiments.

4. Dataset

4.1 CATS — Controlled Anomalies Time Series

The project uses the publicly available:

CATS — Controlled Anomalies Time Series, Version 2

CATS is a multivariate time-series benchmark created for anomaly detection in a complex dynamical system.

It is a **simulated public benchmark**, not a dataset collected from a real factory or physical production facility.

The project therefore makes the following distinction:

Dataset: simulated complex-system telemetry benchmark **Application:** industrial/infrastructure-style monitoring and anomaly detection

The project does not claim that the underlying observations came from a real industrial plant.

4.2 Dataset Characteristics

Property	Value
Dataset	Controlled Anomalies Time Series (CATS)
Version	Version 2
Total observations	5,000,000
Sampling frequency	1 Hz
Time range	2023-01-01 00:00:00 to 2023-02-27 20:53:19 UTC
System variables	17
Ground-truth label columns	2
Anomaly segments	200
Anomaly categories	14
Initial nominal observations	1,000,000
Remaining observations	4,000,000 containing normal and anomalous behaviour
Timestamp	Strictly increasing
Original timestamp gaps	None
Original duplicate timestamps	None

The dataset contains precisely annotated anomalous segments, allowing the project to evaluate not only point-level classification but also event-level detection and detection delay.

4.3 Variable Groups

The dataset contains 17 system variables grouped into three broad categories:

Variable group	Count	Role
Actuation / control commands	4	System control inputs
Environmental stimuli / external forces	3	External influences on the system
Telemetry readings	10	Observed system behaviour
Total	17	

Telemetry variables include measurements such as:

- position
- temperature
- pressure
- voltage
- current
- humidity
- velocity
- acceleration

The exact variable names and semantics used by the project will be documented in the dataset audit and data dictionary rather than inferred from column names alone.

4.4 Ground-Truth Labels

CATS provides two important ground-truth concepts:

y

Binary anomaly label:

```
0 = normal
1 = anomaly
```

category

Anomaly category identifier.

The project will use these labels **for evaluation**, not as input features to the anomaly detectors.

This distinction is essential.

Ground-truth anomaly information must never be available to the detector during inference.

4.5 Anomaly Metadata

The dataset also provides anomaly-segment metadata containing fields such as:

- `start_time`
- `end_time`
- `root_cause`
- `affected`
- `category`

This information allows the project to perform event-level analysis.

For example:

```
Anomaly event
  |
  +-- Start time
  +-- End time
  +-- Category
  +-- Root cause
  +-- Affected variables
```

This makes it possible to investigate not only:

“Did the detector identify an anomalous point?”

but also:

“Did the detector identify the anomalous event, how quickly, and which variables were associated with it?”

4.6 Normal Training Period

The first 1,000,000 observations are nominal.

The project uses this property to establish a clean normal-behaviour reference:

```
Observations 0 → 999,999
  |
  v
Nominal behaviour
  |
  v
Normal-behaviour learning / calibration
```

The remaining observations contain normal and anomalous behaviour and are used for evaluation according to the project’s temporal evaluation contract.

4.7 Missing Data

The original CATS dataset is clean and does not provide naturally occurring missing telemetry as one of its principal anomaly conditions.

Therefore, the project will **not claim that CATS contains naturally occurring missing-data failures**.

Instead, missing-data robustness will be evaluated through a controlled robustness experiment:

```
Original evaluation data
  |
  v
Controlled masking of telemetry
  |
  v
Detector
```

```
|
v
Performance degradation analysis
```

The original dataset remains unchanged.

The experiment tests whether the deployed monitoring pipeline remains useful when telemetry becomes incomplete.

4.8 Dataset Integrity Rules

The following rules apply throughout the project:

1. Original raw data is never modified.
 2. Ground-truth labels are excluded from detector inputs.
 3. Anomaly metadata is used only for evaluation and analysis.
 4. Temporal ordering is preserved.
 5. No future observations are used to construct past monitoring decisions.
 6. Preprocessing parameters are learned only from permitted historical data.
 7. Thresholds are selected using designated validation data.
 8. Final test data remains untouched until final evaluation.
 9. Every experiment records the dataset version and configuration.
 10. Any controlled robustness corruption is documented separately from the original dataset.
-

5. Project Structure

```
TELEMETRY-ANOMALY-DETECTION/
|
+-- data/
|   +-- raw/
|       |   +-- cats/
|       |       +-- telemetry/
|       |       +-- metadata.csv
|       |
|   +-- processed/
|       +-- audit/
|       +-- features/
|       +-- evaluation/
|       +-- robustness/
|
+-- notebooks/
|   +-- 01_problem_and_data_audit.ipynb
|   +-- 02_eda_and_normal_behaviour.ipynb
|   +-- 03_rule_based_detector.ipynb
|   +-- 04_rolling_statistical_detector.ipynb
|   +-- 05_isolation_forest.ipynb
|   +-- 06_evaluation_and_event_analysis.ipynb
|   +-- 07_missing_data_robustness.ipynb
```

```

|   +-- 08_drift_analysis.ipynb
|   +-- 09_final_model_selection.ipynb
|
+-- src/
|   +-- data/
|   |   +-- ingestion.py
|   |   +-- validation.py
|   |   +-- preprocessing.py
|   |
|   +-- detection/
|   |   +-- rule_based.py
|   |   +-- rolling_statistical.py
|   |   +-- isolation_forest.py
|   |   +-- event_detection.py
|   |
|   +-- evaluation/
|   |   +-- point_metrics.py
|   |   +-- event_metrics.py
|   |   +-- detection_delay.py
|   |   +-- robustness.py
|   |   +-- drift.py
|   |
|   +-- monitoring/
|   |   +-- logger.py
|   |   +-- schemas.py
|   |
|   +-- api/
|   |   +-- main.py
|   |   +-- schemas.py
|   |   +-- detector_service.py
|
+-- models/
|   +-- isolation_forest/
|   +-- reproducibility.json
|
+-- reports/
|   +-- figures/
|   +-- tables/
|   +-- anomaly_analysis.md
|   +-- model_card.md
|   +-- experiment_summary.md
|
+-- docs/
|   +-- architecture.png
|   +-- anomaly_taxonomy.md
|   +-- evaluation_contract.md
|   +-- monitoring_runbook.md
|   +-- postmortem_01.md
|

```

```
+-- tests/
|   +-- test_data_validation.py
|   +-- test_detectors.py
|   +-- test_event_detection.py
|   +-- test_evaluation.py
|   +-- test_api.py
|   +-- test_monitoring.py
|
+-- configs/
|   +-- project.yaml
|   +-- evaluation.yaml
|   +-- robustness.yaml
|
+-- backend/
|   +-- Dockerfile
|
+-- .github/
|   +-- workflows/
|       +-- ci.yml
|
+-- README.md
+-- requirements.txt
+-- .env.example
+-- .gitignore
+-- docker-compose.yml
```

6. Setup & Installation

6.1 Prerequisites

- Python 3.10+
- Git
- Docker
- Docker Compose
- Sufficient storage for the CATS dataset
- Sufficient memory for processing large time-series data

The complete 5-million-row dataset is substantially larger than a typical classroom CSV. The project therefore uses chunked or efficient processing where appropriate rather than assuming the entire dataset must always be loaded into memory at once.

6.2 Clone the Repository

```
git clone <repo-url>
cd TELEMETRY-ANOMALY-DETECTION
```

6.3 Create a Virtual Environment

Windows

```
python -m venv .venv
.venv\Scripts\activate
```

Linux / macOS

```
python3 -m venv .venv
source .venv/bin/activate
```

6.4 Install Dependencies

```
pip install -r requirements.txt
```

6.5 Configure Environment Variables

Copy the example environment file:

```
cp .env.example .env
```

On Windows PowerShell:

```
Copy-Item .env.example .env
```

Environment variables may include:

```
LOG_LEVEL=INFO
API_HOST=0.0.0.0
API_PORT=8000
MODEL_PATH=models/isolation_forest/model.joblib
```

Secrets must not be committed to Git.

6.6 Dataset Acquisition

The CATS dataset is publicly available through the project documentation and public dataset repositories.

The repository does not commit the full raw dataset to Git because of its size.

After downloading the dataset, place the source files under:

```
data/raw/cats/
```

The exact expected filenames and paths are recorded in the project configuration.

6.7 Start Jupyter

```
jupyter lab
```

6.8 Run the API Locally

```
uvicorn src.api.main:app --reload --port 8000
```

Health check:

```
curl http://localhost:8000/health
```

Expected response:

```
{  
  "status": "ok"  
}
```

6.9 Run Tests

```
pytest
```

6.10 Run with Docker

```
docker compose up --build
```

The API will be exposed on the configured application port.

7. Methodology

The project follows the monitoring lifecycle:

```
Problem definition  
    |  
    v  
Dataset audit  
    |  
    v  
Temporal validation  
    |  
    v  
Normal behaviour analysis  
    |  
    v  
Rule-based baseline  
    |
```

```

      v
Rolling statistical detector
      |
      v
Isolation Forest
      |
      v
Threshold / sensitivity analysis
      |
      v
Point-level evaluation
      |
      v
Event-level evaluation
      |
      v
Detection-delay analysis
      |
      v
Missing-data robustness
      |
      v
Drift analysis
      |
      v
Error analysis
      |
      v
Final detector selection
      |
      v
API implementation
      |
      v
Monitoring logs
      |
      v
Testing
      |
      v
Deployment

```

7.1 Step 1 — Problem Definition

Define:

- what constitutes normal behaviour;
- what constitutes an anomaly;
- what the detector observes;

- what information is available at detection time;
 - what action follows an alert;
 - what constitutes a useful detection;
 - what constitutes an unacceptable false alarm;
 - how detection delay is measured.
-

7.2 Step 2 — Data Audit

The initial audit examines:

- row count;
- column count;
- timestamp range;
- sampling interval;
- duplicate timestamps;
- missing timestamps;
- missing values;
- variable types;
- constant columns;
- unique values;
- anomaly prevalence;
- anomaly segment lengths;
- category distribution;
- affected-variable information;
- root-cause metadata.

The audit is completed before model selection.

7.3 Step 3 — Temporal Integrity Audit

The monitoring system must respect temporal order.

The audit verifies:

```
t0 < t1 < t2 < ... < tn
```

and checks that the expected sampling interval is maintained.

For CATS, the documented sampling frequency is 1 Hz.

The project will independently verify this property during the data audit rather than simply trusting the documentation.

7.4 Step 4 — Normal Behaviour Analysis

The first nominal portion of CATS provides a reference period for understanding normal behaviour.

Analysis includes:

- distribution of each variable;
- mean and standard deviation;

- quantiles;
- autocorrelation;
- temporal stability;
- cross-variable relationships;
- rolling statistics;
- correlation structure;
- variable scale differences;
- unusual operating regimes.

The purpose is not merely exploratory plotting.

The purpose is to understand:

What does normal system behaviour actually look like?

7.5 Step 5 — Leakage Audit

The detector must not use information that would only become available after the anomaly has already occurred.

Forbidden information during inference includes:

- future observations;
- future labels;
- anomaly category;
- root-cause metadata;
- affected-variable ground truth;
- future rolling statistics;
- manually derived information from the complete anomaly segment.

The ground-truth labels are used only after predictions have been generated.

7.6 Step 6 — Rule-Based Baseline

The first detector is deliberately simple.

Possible rules include:

```
value outside normal operating range
```

or:

```
absolute deviation from normal baseline > threshold
```

The exact rule will be selected after the normal-behaviour audit.

The baseline establishes the performance and operational burden of a simple monitoring system.

7.7 Step 7 — Rolling Statistical Detector

The second detector adapts to recent system behaviour.

A general form is:

$$z_t = \frac{x_t - \mu_t}{\sigma_t}$$

where:

- x_t is the current observation;
- μ_t is a rolling estimate of the recent mean;
- σ_t is a rolling estimate of recent variability.

An observation can be flagged when:

$$|z_t| > k$$

for a selected threshold k .

The actual window size and threshold will be selected through validation rather than assumed in advance.

7.8 Step 8 — Isolation Forest

Isolation Forest provides an unsupervised machine-learning detector.

The central idea is that unusual observations can often be isolated using fewer random partitioning operations than normal observations.

The project will investigate:

- contamination assumptions;
- number of estimators;
- maximum samples;
- random seed;
- feature scaling requirements;
- score distributions;
- threshold selection.

Hyperparameters will be evaluated systematically rather than chosen solely because they are common defaults.

7.9 Step 9 — Threshold Analysis

Anomaly detectors often produce a continuous score before producing a binary alert.

Therefore:

```
Telemetry
  ↓
Anomaly score
  ↓
Threshold
  ↓
Normal / Anomaly
```

The threshold determines the trade-off between:

- missed anomalies;

- false alarms;
- detection sensitivity.

Thresholds will be selected using validation data and then frozen before final test evaluation.

7.10 Step 10 — Point-Level Evaluation

Each timestamp can be evaluated against the ground-truth anomaly label.

The basic confusion matrix is:

	Actual Normal	Actual Anomaly
Predicted Normal	TN	FN
Predicted Anomaly	FP	TP

Point-level metrics include:

- Precision
- Recall
- F1
- False-positive rate
- False-negative rate

Point-level metrics are useful but are **not sufficient by themselves**.

7.11 Step 11 — Event-Level Evaluation

Operational anomalies are treated as events rather than merely independent rows.

A ground-truth anomaly segment has:

```
start_time
end_time
category
root_cause
affected_variables
```

The detector's point-level alerts are grouped into temporal events.

The evaluation then asks:

Did the detector identify the anomaly event at all?

This avoids treating a 60-second anomaly as 60 completely independent business incidents.

7.12 Step 12 — Detection Delay

For each detected anomaly event:

$$\text{Detection Delay} = \text{First Detection Time} - \text{Anomaly Start Time}$$

A detector that eventually identifies every anomaly but responds extremely late may be less useful than a detector with slightly lower recall but substantially faster detection.

Detection delay will therefore be reported using appropriate summary statistics such as:

- mean;
 - median;
 - P90;
 - maximum;
 - category-level delay.
-

7.13 Step 13 — False-Alarm Analysis

False alarms are measured because excessive alerts can create alert fatigue.

The project will report metrics such as:

$$\text{False Positive Rate} = \frac{FP}{FP + TN}$$

and, where appropriate, an operational false-alarm frequency such as:

$$\text{False Alarm Rate} = \frac{\text{Number of false alert events}}{\text{Amount of monitored time}}$$

The exact operational definition will be fixed before final comparison.

7.14 Step 14 — Missing-Data Robustness

Because the original CATS signal is clean, a controlled evaluation will test the detector under missing telemetry.

Examples include:

- isolated missing observations;
- short contiguous gaps;
- longer telemetry outages;
- missing values in selected variables;
- increased missingness rates.

The original data remains unchanged.

The experiment measures:

```
Clean performance
      ↓
Performance with missing telemetry
      ↓
Performance degradation
```

7.15 Step 15 — Drift Analysis

Monitoring systems operate in environments where normal behaviour can change.

Drift analysis investigates whether detector performance changes when the distribution of telemetry changes over time.

The project will examine:

- feature distribution changes;
- rolling mean changes;
- variance changes;
- anomaly-score distribution changes;
- false-alarm changes;
- recall changes;
- detection-delay changes.

The goal is not to assume that every distribution change is an anomaly.

A change in normal operating behaviour can itself be a legitimate regime change.

7.16 Step 16 — Error Analysis

Every false positive and false negative should be treated as an opportunity to understand detector behaviour.

Questions include:

- Which variables were involved?
 - Which anomaly categories were difficult?
 - Were anomalies gradual or abrupt?
 - Were anomalies short-lived?
 - Did several variables change simultaneously?
 - Did the detector require a large deviation from baseline?
 - Did missing data contribute to the failure?
 - Did the detector generate repeated alerts?
 - Did normal drift resemble an anomaly?
-

7.17 Step 17 — Final Detector Selection

The final detector is not selected using a single metric.

The decision considers:

```
Detection quality
      +
False-alarm burden
      +
Detection delay
      +
Robustness
      +
Computational cost
```

+
Operational complexity

A more complicated model is accepted only if the measured improvement justifies its additional complexity.

8. Baselines

Golden Rule: Never claim an improvement that has not been measured against a baseline.

8.1 Baseline 1 — Rule-Based Detector

Field	Value
Detector	Rule-based
Purpose	Simple operational reference
Training	Normal reference period
Primary measures	Recall, false alarms, detection delay
Status	To be measured

8.2 Baseline 2 — Rolling Statistical Detector

Field	Value
Detector	Rolling statistical
Purpose	Adaptive statistical reference
Training	Historical normal behaviour
Primary measures	Recall, false alarms, detection delay
Status	To be measured

8.3 Candidate 3 — Isolation Forest

Field	Value
Detector	Isolation Forest
Type	Unsupervised ML
Input	Telemetry features
Ground-truth labels during inference	No
Primary measures	Recall, precision, false alarms, delay
Status	To be measured

8.4 Baseline Recording Card

Field	Value
Detector	Rule-based
Dataset	CATS Version 2
Normal reference period	First nominal period
Evaluation strategy	Time-ordered
Primary metric	Event detection / operational metric
Precision	<i>to be recorded</i>
Recall	<i>to be recorded</i>
False-alarm rate	<i>to be recorded</i>
Median detection delay	<i>to be recorded</i>
Date recorded	<i>to be recorded</i>

9. Experiments

Experiments are tracked so that detector comparisons remain reproducible.

Each experiment should record:

- dataset version;
- dataset hash where practical;
- code commit;
- Python version;
- dependency versions;
- detector type;
- feature set;
- window size;
- threshold;
- random seed;
- evaluation period;
- metrics;
- robustness condition.

9.1 Experiment Matrix

Experiment	Question
E01	What does normal system behaviour look like?
E02	How well does the rule-based detector perform?
E03	How well does the rolling statistical detector perform?
E04	How well does Isolation Forest perform?
E05	How does threshold selection affect false alarms and recall?
E06	Which detector detects anomaly events most reliably?
E07	Which detector detects anomalies fastest?
E08	Which anomaly categories are hardest?
E09	How does missing telemetry affect performance?

Experiment	Question
E10	How does distributional drift affect performance?
E11	Which variables contribute most to difficult detections?
E12	Does ML materially outperform simpler monitoring methods?

9.2 One-Variable-at-a-Time Rule

Unless the experiment is explicitly designed as a multi-parameter search:

Change one meaningful variable at a time.

Examples:

- detector;
- threshold;
- rolling window;
- feature set;
- Isolation Forest hyperparameter.

This makes causal interpretation of experiment results easier.

9.3 Experiment Tracking

Each experiment should produce a record containing:

```

experiment_id
timestamp
git_commit
dataset_version
dataset_hash
detector
parameters
features
evaluation_window
precision
recall
f1
false_alarm_rate
detection_delay
robustness_condition
notes

```

9.4 Optimization Decision Matrix

Dimension	Baseline	Candidate	Change	Accept?
Precision	—	—	—	—
Recall	—	—	—	—

Dimension	Baseline	Candidate	Change	Accept?
False-alarm rate	—	—	—	—
Detection delay	—	—	—	—
Missing-data degradation	—	—	—	—
Drift degradation	—	—	—	—
Runtime	—	—	—	—

10. Results

To be completed after experiments are executed. No result is entered without measurement.

10.1 Overall Detector Comparison

Detector	Precision	Recall	F1	False-Alarm Rate	Median Detection Delay
Rule-based	—	—	—	—	—
Rolling statistical	—	—	—	—	—
Isolation Forest	—	—	—	—	—

10.2 Event-Level Results

Detector	Events	Events Detected	Event Detection Rate	Median Delay	P90 Delay
Rule-based	—	—	—	—	—
Rolling statistical	—	—	—	—	—
Isolation Forest	—	—	—	—	—

10.3 Anomaly Category Results

Category	Detector	Precision	Recall	Event Detection	Median Delay
Category 0	—	—	—	—	—
Category 1	—	—	—	—	—
Category 2	—	—	—	—	—
...	...	...	...	...	...
Category 13	—	—	—	—	—

The actual anomaly-category meanings will be documented from the CATS metadata rather than inferred from category numbers.

10.4 Missing-Data Robustness

Missingness Condition	Detector	Recall	False-Alarm Rate	Delay	Performance Change
Clean	—	—	—	—	—
Low missingness	—	—	—	—	—
Moderate missingness	—	—	—	—	—
High missingness	—	—	—	—	—

10.5 Drift Results

Detector	Baseline Recall	Drift Recall	Recall Change	Baseline FAR	Drift FAR
Rule-based	—	—	—	—	—
Rolling statistical	—	—	—	—	—
Isolation Forest	—	—	—	—	—

11. Error Analysis

11.1 False Positives

For false-positive events, investigate:

- variable involved;
- magnitude of deviation;
- duration;
- operating regime;
- preceding system behaviour;
- whether multiple variables changed;
- whether the detector was overly sensitive;
- whether the event represented normal but unusual behaviour.

11.2 False Negatives

For missed anomaly events, investigate:

- anomaly duration;
- anomaly magnitude;
- affected variables;
- anomaly category;
- whether the anomaly was gradual;
- whether the detector required a larger deviation;
- whether the anomaly was masked by normal system variability;
- whether missing data contributed to the failure.

11.3 Error Slicing

Performance will be examined by:

Slice	Purpose
Anomaly category	Identify difficult anomaly types
Affected variable	Identify weak telemetry channels
Root cause	Identify systematic detector weaknesses
Event duration	Compare short vs. long anomalies
Anomaly magnitude	Compare subtle vs. severe deviations
Temporal period	Identify performance changes over time
Missingness level	Measure robustness
Drift regime	Measure adaptation problems

11.4 Detection Delay Analysis

Detection delay will be analysed by anomaly:

```
Anomaly event
|
+-- Start
+-- First detector alert
+-- Delay
+-- End
```

Long-delay events will receive individual investigation.

11.5 Root-Cause and Affected-Variable Analysis

CATS metadata provides information about root cause and affected variables.

This allows the project to investigate:

Which system variables were involved in the anomaly, and did the detector respond to those changes?

This is an analytical aid, not a claim that the detector itself performs causal diagnosis.

The project must distinguish:

- **anomaly detection**
- **root-cause analysis**

A detector can identify abnormal behaviour without correctly identifying the underlying physical or operational cause.

12. Deployment

12.1 Backend — FastAPI

The detector will be exposed through a lightweight API.

Endpoint	Method	Description
/health	GET	API health check
/metadata	GET	Detector/model metadata
/predict	POST	Evaluate telemetry and return anomaly score/decision
/events	GET	Retrieve recent logged anomaly events
/metrics	GET	Monitoring summary

12.2 Example /predict Response

A prediction response may contain:

```
{
  "timestamp": "2023-02-01T12:00:00Z",
  "anomaly_score": 0.84,
  "is_anomaly": true,
  "detector_version": "isolation-forest-v1"
}
```

The exact response schema will be defined by the API contract and implemented with Pydantic.

12.3 Input Validation

The API validates:

- required telemetry fields;
- timestamp format;
- numeric types;
- missing values;
- acceptable value ranges where justified;
- detector compatibility.

Invalid requests should return appropriate HTTP validation errors rather than reaching the model unchecked.

12.4 Monitoring Logs

Each detector decision should be traceable.

A monitoring log may contain:

```
event_id
timestamp
detector_version
anomaly_score
```

```
is_anomaly
event_status
processing_time_ms
```

The system must avoid logging unnecessary personal or sensitive information.

12.5 Event Aggregation

Individual anomalous timestamps may be grouped into a monitoring event.

For example:

```
12:00:01 anomaly
12:00:02 anomaly
12:00:03 anomaly
12:00:04 anomaly
```

may represent one operational event rather than four separate alerts.

The event aggregation policy will be explicitly documented and tested.

12.6 Containerisation

```
docker compose up --build
```

Expected services:

```
backend
monitoring/logging service
```

A database is optional and should only be introduced where it provides a clear operational benefit.

12.7 CI/CD — GitHub Actions

The CI workflow should run on pushes and pull requests.

Job	Command / Purpose
Test	<code>pytest</code>
Lint	<code>ruff</code>
Import check	Validate package imports
API smoke test	Start API and call <code>/health</code>
Build	Build Docker image

12.8 Deployment Target

The API may be deployed to a suitable cloud platform.

The README will record the actual deployment platform only after deployment has been completed.

No deployment status will be claimed before it is verified.

13. Evaluation Contract

This section turns “the detector seems good” into an engineering evaluation contract.

Field	Contract
Dataset	CATS Version 2
Problem	Multivariate telemetry anomaly detection
Evaluation type	Time-aware
Point-level metrics	Precision, Recall, F1, false-positive rate
Event-level metrics	Event detection rate, missed events
Operational metric	Detection delay
Robustness	Missing telemetry and drift
Baseline 1	Rule-based detector
Baseline 2	Rolling statistical detector
ML candidate	Isolation Forest
Threshold selection	Validation data only
Final test	Untouched until final evaluation

13.1 Acceptance Principle

There is deliberately **no arbitrary universal threshold such as “F1 must be 0.70.”**

Anomaly-detection performance depends on:

- anomaly prevalence;
- anomaly definition;
- event duration;
- false-alarm tolerance;
- detection-delay requirements;
- operating context.

The final detector must therefore be accepted based on a documented trade-off against the baselines.

13.2 Final Selection Rule

A detector is preferred when it provides a defensible improvement in:

1. event detection;
2. detection delay;
3. false-alarm burden;
4. robustness;

without introducing disproportionate:

- computational cost;
- implementation complexity;

- maintenance burden.
-

13.3 Failure Criteria

An experiment requires investigation when:

- false alarms increase substantially without meaningful recall improvement;
 - detection delay becomes operationally excessive;
 - robustness collapses under modest missingness;
 - performance degrades substantially under drift;
 - the detector performs substantially worse than the simpler baseline;
 - results cannot be reproduced;
 - leakage is discovered.
-

14. Limitations

14.1 Simulated Benchmark

CATS is a simulated complex-system benchmark.

Although it is useful for developing and evaluating anomaly-detection methods, its behaviour should not be interpreted as direct evidence of performance on a particular real-world factory, power plant, water-treatment facility, or other production environment.

14.2 Dataset-Specific Anomalies

The anomaly types and patterns represented in CATS are specific to the benchmark.

A detector that performs well on CATS may not automatically perform well on:

- real sensor noise;
 - equipment wear;
 - unlabelled failures;
 - sensor calibration errors;
 - new failure modes;
 - real operational regime changes.
-

14.3 Missing Data

The original dataset is clean.

Missing-data robustness therefore comes from a controlled evaluation experiment rather than naturally occurring missingness.

This is useful for testing robustness but is not equivalent to validating the system against naturally occurring industrial telemetry outages.

14.4 Drift

Controlled drift experiments cannot reproduce every form of real-world concept drift.

Production systems can change because of:

- equipment replacement;
 - maintenance;
 - seasonal conditions;
 - control-policy changes;
 - sensor recalibration;
 - process redesign;
 - changes in operating loads.
-

14.5 Ground Truth

CATS provides precise anomaly labels because anomalies were deliberately introduced.

Real operational environments often have:

- delayed incident labels;
- incomplete labels;
- disputed incident boundaries;
- unknown anomalies;
- undocumented failures.

Therefore, real-world evaluation can be considerably harder.

14.6 Anomaly Detection Is Not Failure Prediction

This project detects abnormal behaviour.

It does not directly predict:

- remaining useful life;
- exact failure time;
- maintenance cost;
- equipment replacement date.

A detected anomaly is a **signal for investigation**, not proof that a component will fail.

14.7 Root Cause Is Not Automatically Identified

Although CATS provides root-cause and affected-variable metadata for analysis, the anomaly detector itself is not assumed to perform causal diagnosis.

Root-cause identification would require a separate methodology and evaluation contract.

15. Security Considerations

15.1 API Input Validation

All API input should be validated before reaching the detector.

Invalid:

- timestamps;
- numeric values;
- missing required fields;
- malformed payloads;

should be rejected.

15.2 Environment Variables

Credentials and deployment configuration must be stored in environment variables.

Never commit:

```
.env
API keys
cloud credentials
database passwords
private tokens
```

to Git.

15.3 Logging

Logs should contain only information necessary for monitoring and debugging.

The monitoring service should avoid storing:

- secrets;
 - credentials;
 - unnecessary personal information;
 - raw payloads when they are not required.
-

15.4 Dependency Security

Dependencies should be pinned or constrained appropriately and reviewed for known vulnerabilities.

15.5 Model Integrity

The deployed detector should be associated with:

- model version;
- Git commit;
- dataset version;

- configuration;
- dependency versions.

This prevents an unexplained change in detector behaviour from being attributed incorrectly to the underlying data.

16. Future Work

Priority	Item
High	Complete full CATS data audit
High	Document all anomaly categories from source metadata
High	Complete rule-based baseline
High	Complete rolling statistical detector
High	Train and evaluate Isolation Forest
High	Complete event-level evaluation
High	Measure detection delay
High	Complete missing-data robustness experiments
High	Complete drift experiments
High	Build FastAPI detector service
High	Implement monitoring logs
High	Write full pytest suite
Medium	Add anomaly visualisation dashboard
Medium	Add detector calibration / threshold study
Medium	Add alert deduplication
Medium	Add detector health monitoring
Medium	Add automated drift alerts
Medium	Write deliberate failure postmortem
Medium	Add rollback procedure
Low	Compare additional unsupervised algorithms
Low	Investigate online/streaming anomaly detection
Low	Evaluate on a second public industrial/infrastructure dataset
Low	Investigate explainability techniques
Low	Add production-scale message-stream ingestion

Appendix A — Anomaly Detection Workflow

The complete engineering workflow is:

```

1. Define the operational problem
    |
    v
2. Identify the monitoring user
    |
    v
3. Define normal behaviour
    |

```

```

      v
4. Audit the dataset
    |
    v
5. Verify temporal integrity
    |
    v
6. Establish the normal reference period
    |
    v
7. Build rule-based baseline
    |
    v
8. Measure baseline
    |
    v
9. Build rolling statistical detector
    |
    v
10. Measure statistical detector
    |
    v
11. Build Isolation Forest
    |
    v
12. Tune using validation data
    |
    v
13. Freeze final threshold
    |
    v
14. Evaluate point-level detection
    |
    v
15. Evaluate event-level detection
    |
    v
16. Measure detection delay
    |
    v
17. Measure false-alarm burden
    |
    v
18. Analyse anomaly categories
    |
    v
19. Analyse root-cause / affected variables
    |
    v
```

```
20. Test missing-data robustness
    |
    v
21. Test drift robustness
    |
    v
22. Compare against baselines
    |
    v
23. Select final detector
    |
    v
24. Build API
    |
    v
25. Implement monitoring logs
    |
    v
26. Test
    |
    v
27. Containerise
    |
    v
28. Deploy
    |
    v
29. Monitor
    |
    v
30. Document failures and improvements
```

Appendix B — Final Project Scorecard

Problem Definition

- ☐ Operational monitoring problem defined
- ☐ Stakeholders identified
- ☐ Normal behaviour defined
- ☐ Anomaly definition documented
- ☐ Monitoring action documented
- ☐ False-alarm implications documented
- ☐ Detection-delay objective documented

Dataset

- ☐ CATS Version 2 documented
- ☐ Dataset provenance documented

- ☐ Dataset integrity verified
- ☐ Timestamp ordering verified
- ☐ Sampling interval verified
- ☐ Missing-value audit completed
- ☐ Duplicate audit completed
- ☐ Anomaly prevalence measured
- ☐ Anomaly segments verified
- ☐ Category metadata documented
- ☐ Root-cause metadata documented
- ☐ Affected-variable metadata documented

Leakage

- ☐ Ground-truth labels excluded from detector inputs
- ☐ Anomaly metadata excluded from inference
- ☐ Future observations excluded from historical decisions
- ☐ Rolling features use only permitted historical information
- ☐ Threshold selected without using final test results
- ☐ Final test set remains untouched until final evaluation

Baselines

- ☐ Rule-based detector implemented
- ☐ Rule-based detector evaluated
- ☐ Rolling statistical detector implemented
- ☐ Rolling statistical detector evaluated
- ☐ Baseline results recorded

Machine Learning

- ☐ Isolation Forest implemented
- ☐ Hyperparameters documented
- ☐ Validation procedure documented
- ☐ Threshold selection documented
- ☐ Random seed recorded
- ☐ Final detector version recorded

Evaluation

- ☐ Precision calculated
- ☐ Recall calculated
- ☐ F1 calculated
- ☐ False-positive rate calculated
- ☐ False-alarm rate calculated
- ☐ Point-level evaluation completed
- ☐ Event-level evaluation completed
- ☐ Detection delay calculated
- ☐ Detection delay distribution analysed
- ☐ Anomaly-category analysis completed
- ☐ Affected-variable analysis completed
- ☐ Error analysis completed

Robustness

- ☐ Missing-data experiment completed
- ☐ Short missing gaps tested
- ☐ Longer missing gaps tested
- ☐ Missing-data performance degradation measured
- ☐ Drift experiment completed
- ☐ Drift performance degradation measured
- ☐ False-alarm behaviour under drift analysed

Reproducibility

- ☐ Git commit recorded
- ☐ Dataset version recorded
- ☐ Dataset hash recorded where practical
- ☐ Python version recorded
- ☐ Dependency versions recorded
- ☐ Configuration recorded
- ☐ Random seed recorded
- ☐ Reproducibility manifest produced

Deployment

- ☐ FastAPI service implemented
- ☐ `/health` implemented
- ☐ `/metadata` implemented
- ☐ `/predict` implemented
- ☐ Input validation implemented
- ☐ Event aggregation implemented
- ☐ Monitoring logs implemented
- ☐ API tests written
- ☐ Docker build works
- ☐ CI workflow passes
- ☐ Deployment completed and verified

Monitoring

- ☐ Detector version logged
- ☐ Anomaly score logged
- ☐ Alert decision logged
- ☐ Processing time logged
- ☐ Event IDs generated
- ☐ False-alarm review process documented
- ☐ Drift monitoring design documented
- ☐ Rollback procedure documented
- ☐ Deliberate failure postmortem completed

Documentation

- ☐ README complete
- ☐ Architecture diagram present
- ☐ Dataset audit documented

- ☐ Anomaly taxonomy documented
 - ☐ Evaluation contract documented
 - ☐ Model card written
 - ☐ Monitoring runbook written
 - ☐ Limitations documented
 - ☐ Security considerations documented
 - ☐ Final results reported with measured values only
-

Appendix C — Mathematics Reference

C.1 Precision

$$Precision = \frac{TP}{TP + FP}$$

Interpretation:

Of the observations flagged as anomalous, what proportion were actually anomalous?

C.2 Recall

$$Recall = \frac{TP}{TP + FN}$$

Interpretation:

Of the actual anomalous observations, what proportion did the detector identify?

C.3 F1 Score

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

F1 balances precision and recall.

It should not be treated as the only metric because it does not directly express operational false-alarm burden or detection delay.

C.4 False-Positive Rate

$$FPR = \frac{FP}{FP + TN}$$

Interpretation:

Among normal observations, how often did the detector incorrectly raise an anomaly signal?

C.5 False-Negative Rate

$$FNR = \frac{FN}{FN + TP}$$

Interpretation:

Among anomalous observations, how often did the detector fail to identify the anomaly?

C.6 Rolling Mean

For a window of w observations:

$$\mu_t = \frac{1}{w} \sum_{i=t-w+1}^t x_i$$

In a causal monitoring implementation, the current observation must be handled according to the detector's explicitly defined evaluation policy so that future observations never enter the calculation.

C.7 Rolling Standard Deviation

$$\sigma_t = \sqrt{\frac{1}{w-1} \sum_{i=t-w+1}^t (x_i - \mu_t)^2}$$

C.8 Standardised Deviation

$$z_t = \frac{x_t - \mu_t}{\sigma_t}$$

A threshold can be applied:

$$|z_t| > k$$

where k is selected using validation data.

C.9 Detection Delay

For anomaly event i :

$$D_i = T_{\text{first detection},i} - T_{\text{start},i}$$

where:

- $T_{\text{start},i}$ = true anomaly start time;
 - $T_{\text{first detection},i}$ = first valid detector alert associated with that event.
-

C.10 Mean Detection Delay

$$\bar{D} = \frac{1}{N} \sum_{i=1}^N D_i$$

The mean should be reported alongside robust summaries such as median and P90 because a small number of extremely delayed events can distort the mean.

C.11 Event Detection Rate

$$\text{Event Detection Rate} = \frac{\text{Detected Anomaly Events}}{\text{Total Anomaly Events}}$$

This differs from point-level recall because one anomaly event may contain many anomalous observations.

C.12 False-Alarm Rate

A monitoring-oriented formulation is:

$$\text{False Alarm Rate} = \frac{\text{False Alert Events}}{\text{Monitored Time}}$$

The exact unit — for example, false alerts per hour or per day — must be specified in the evaluation contract.

C.13 Isolation Forest Concept

Isolation Forest assigns an anomaly score based on how easily observations can be isolated through random partitioning.

The intuition is:

```
Unusual observation
      ↓
Easy to isolate
      ↓
Shorter isolation path
      ↓
Higher anomaly evidence
```

Normal observations tend to require more partitions to isolate.

C.14 Drift

For a feature distribution P_t at time t , drift occurs when:

$$P_t(X) \neq P_{\text{reference}}(X)$$

The project distinguishes:

- **data drift** — input distribution changes;
- **behavioural change** — the monitored system changes its normal operating behaviour;
- **anomaly** — behaviour deviates from the expected operating pattern.

These concepts are related but not interchangeable.

C.15 Core Monitoring Principle

The central engineering objective can be summarized as:

Detection Quality + Low False Alarm Burden + Timely Detection + Robustness]

A detector is not considered successful merely because it produces a high classification score.

The final decision must be supported by measured evidence.